

Documentație de Proiectare și Planificare Agile

Proiect: Sistem Inteligent de Monitorizare a Cheltuielilor Familiei

Modul: M4 - Hosting / Infrastructură / CI/CD (DevOps)

Echipa de dezvoltare:

- Scrum Master: FIRTESCU SEBASTIAN-IOAN
- Membru echipa: TARCA GHEORGHITA-CATALIN
- Membru echipa: STUPU EDUARD
- Membru echipa: SESCO GABRIEL TIBERIU
- Membru echipa: GHEOCA RAZVAN

Product backlog:

Epic 1: Infrastructura de baza (DigitalOcean)

- 1.1: Crearea VPS(Droplet) pe DigitalOcean(Ubuntu 24.04 LTS, 4GB RAM)
- 1.2: Configurare securitate server si deschiderea exclusiva a porturilor necesare.
- 1.3: Securizarea accesului la server prin chei SSH, doar pentru membrii M4.

Epic 2: Containerizare (Docker)

- 2.1: Instalare Docker pe VPS
- 2.2: Crearea unui DockerFile optimizat pentru Backend(Spring Boot, Java)
- 2.3: Redactarea fisierului docker-compose.yml pentru gestionarea containerelor: Spring Boot, PostgreSQL, etc.
- 2.4: Configurarea volumelor persistente pentru DB, pentru a asigura retinerea datelor la restartare.

Epic 3: Networking si Reverse Proxy (Nginx)

- 3.1: Configurare Nginx ca Reverse Proxy: Directionarea traficului de pe portul 80 (HTTP) catre containerul de Spring Boot (port 8080).
- 3.2: Implementare SSL/HTTPS: Utilizarea Certbot (Let's Encrypt) pentru securizarea comunicarii si criptarea datelor financiare.
- 3.3: Gestionarea politicilor CORS: Configurarea serverului pentru a permite cererile venite de la domeniul de Frontend.

Epic 4: Automatizare CI/CD (GitHub Actions)

- 4.1: Pipeline de Integrare Continua (CI): Rularea automata a testelor Maven la fiecare Push/Pull Request pentru validarea codului.
- 4.2: Pipeline de Deployment Continuu (CD): Automatizarea procesului de Build imagine Docker si update pe VPS la fiecare push in ramura principala.
- 4.3: Monitorizare si Alerte: Notificarea echipei pe Discord/GitHub in cazul esuarii unui proces de build sau deploy.

Epic 5: Integrare Servicii si Management Secrete

- 5.1: Gestiunea Secretelor: Stocarea cheilor sensibile (Gemini, Google Maps, Cloudinary) in GitHub Secrets si variabile de mediu.
- 5.2: Configurare Cloudinary: Pregatirea mediului de stocare pentru pozele cu extrase de cont si bonuri procesate de modulul OCR.
- 5.3: Deployment Frontend: Conectarea interfetei grafice si asigurarea legaturii cu IP-ul serverului DigitalOcean.

Iteratia 1 - Fundatia sistemului

Durata: 1-2 Saptamani (Finalul lunii martie)

Obiectiv Sprint 1: Setup-ul initial al infrastructurii si validarea accesului la server.

Task-uri Iteratia 1:

1. Achizitionarea si configurarea Droplet-ului pe DigitalOcean.
2. Securizarea accesului prin SSH si setarea Firewall-ului.
3. Instalarea Docker si Docker Compose pe server.
4. Rularea unui container Nginx pentru a valida comunicarea cu exteriorul.
5. Crearea structurii initiale de foldere pentru volumele bazei de date.

Definition of Done (DoD) :

- Serverul este online si accesibil prin SSH.
- Docker ruleaza fara erori pe VPS.
- Exista un fisier **docker-compose.yml** de baza urcat pe GitHub.
- Adresa IP a serverului este comunicata celorlalte echipe pentru teste de conectivitate.

Diagrame:

- Diagrama C4 - Gheoca Razvan

- C4 level 2->diagrama de containere
 - specifici cum sunt impartite piesele software pe server

Boundary: Sistemul Family Agent(VPS-ul)

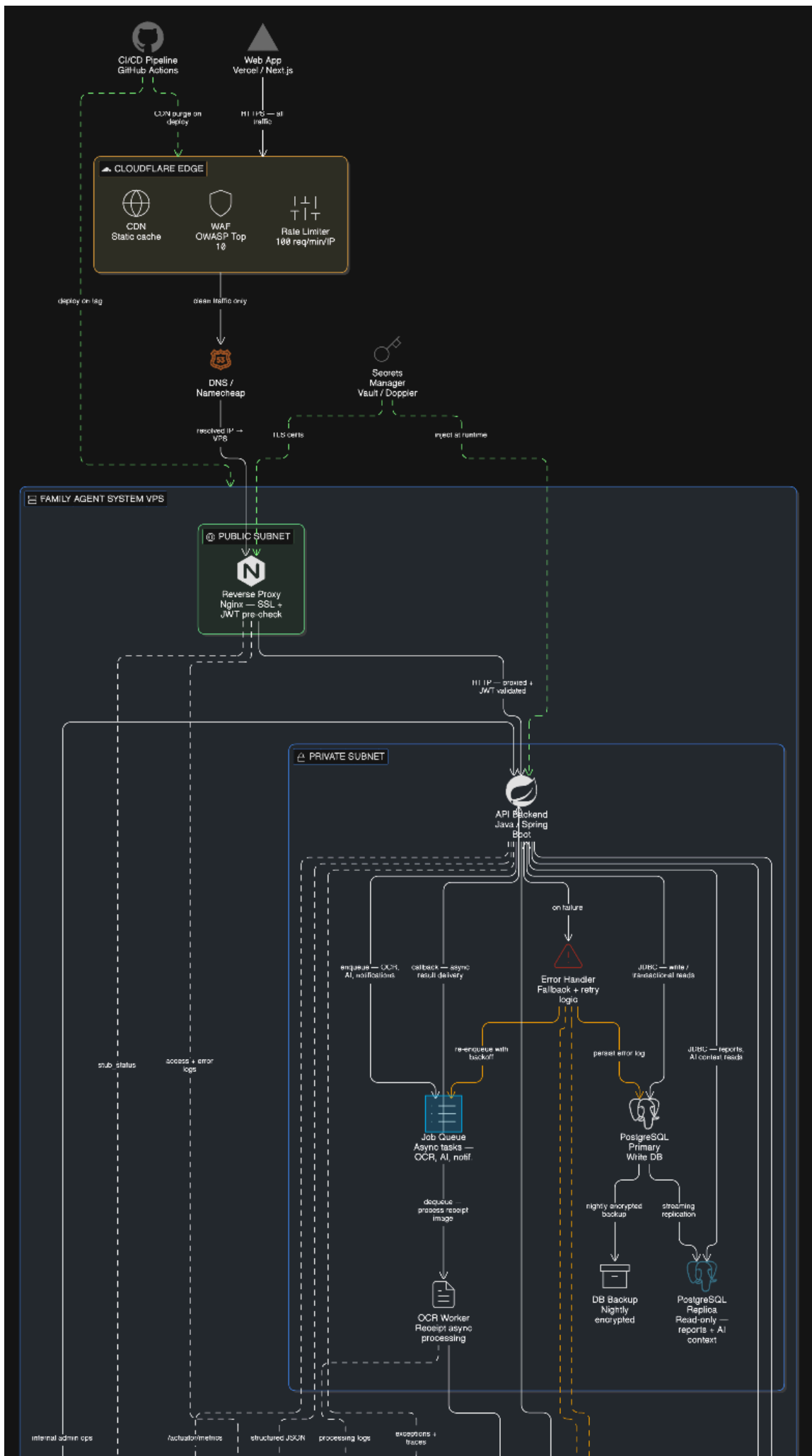
Container 1:Nginx(Reverse Proxy) - securitate + redirectionare.

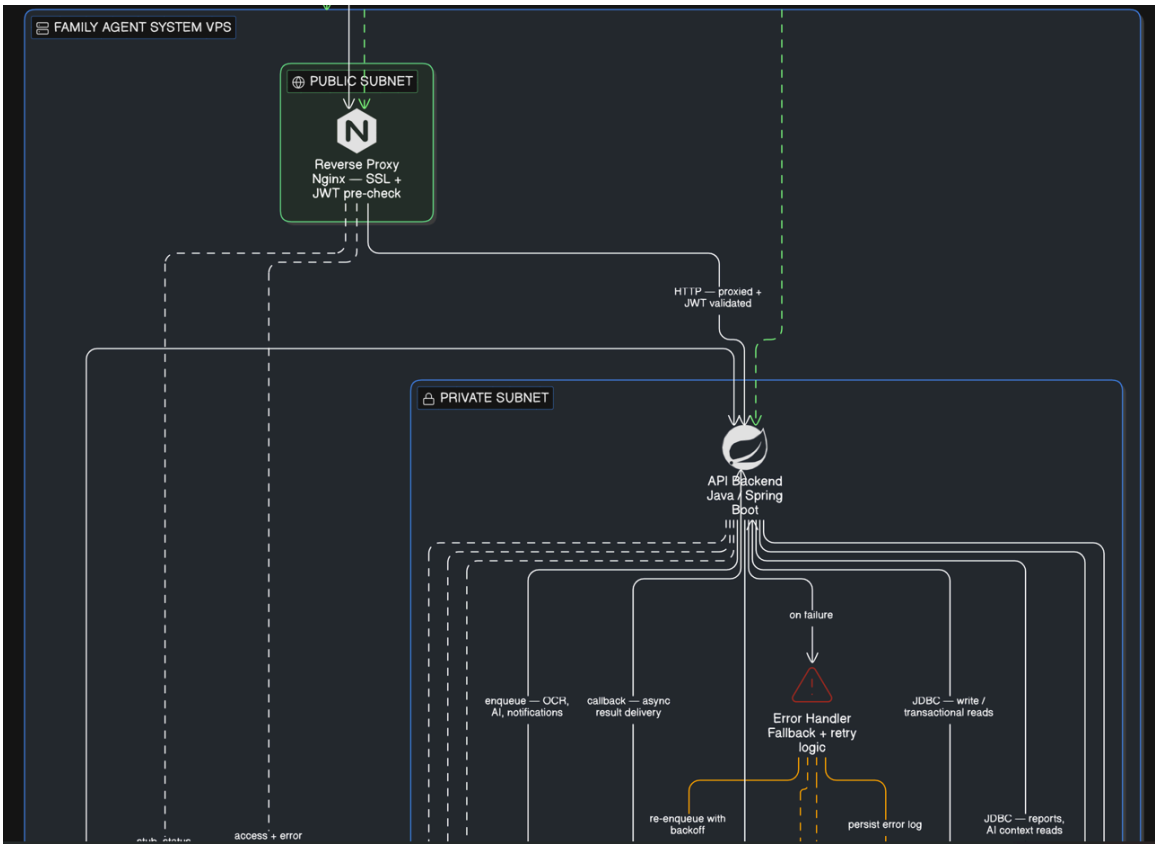
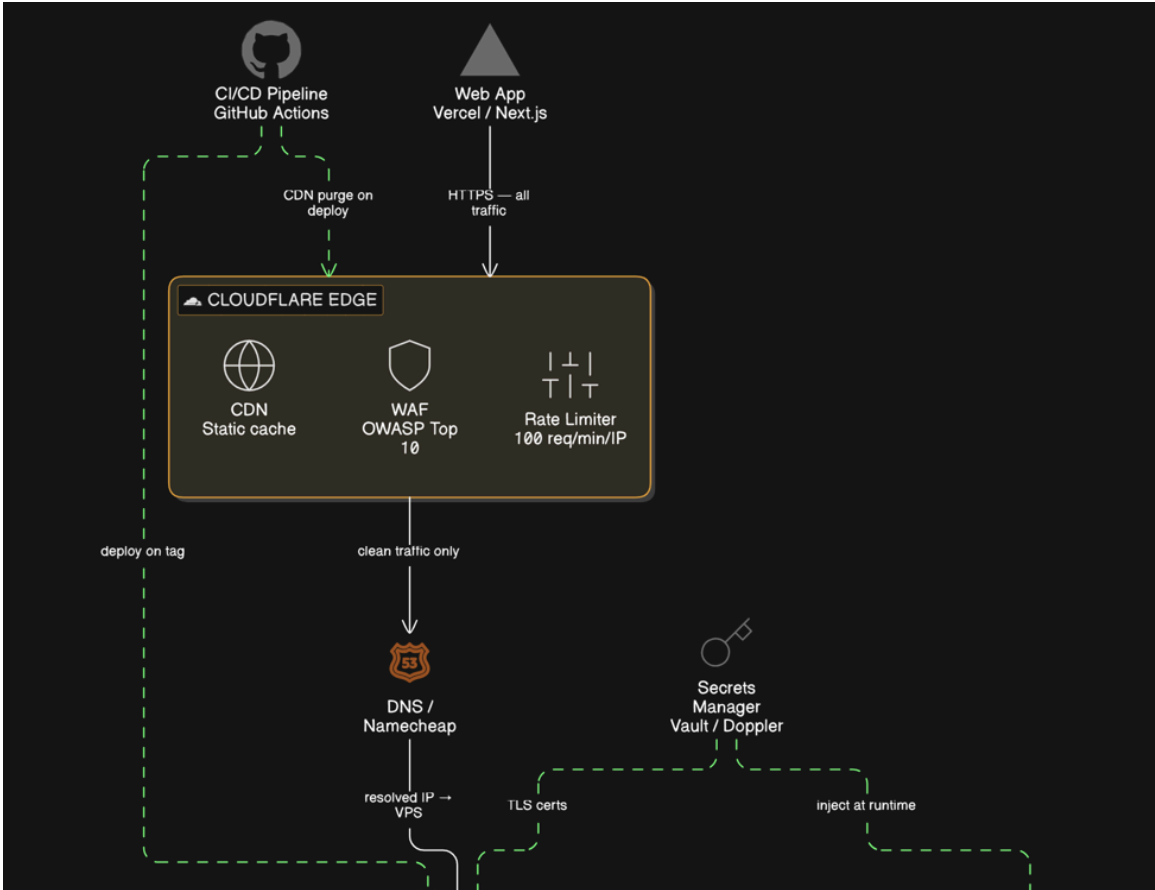
Container2: Spring Boot App(API Backend)

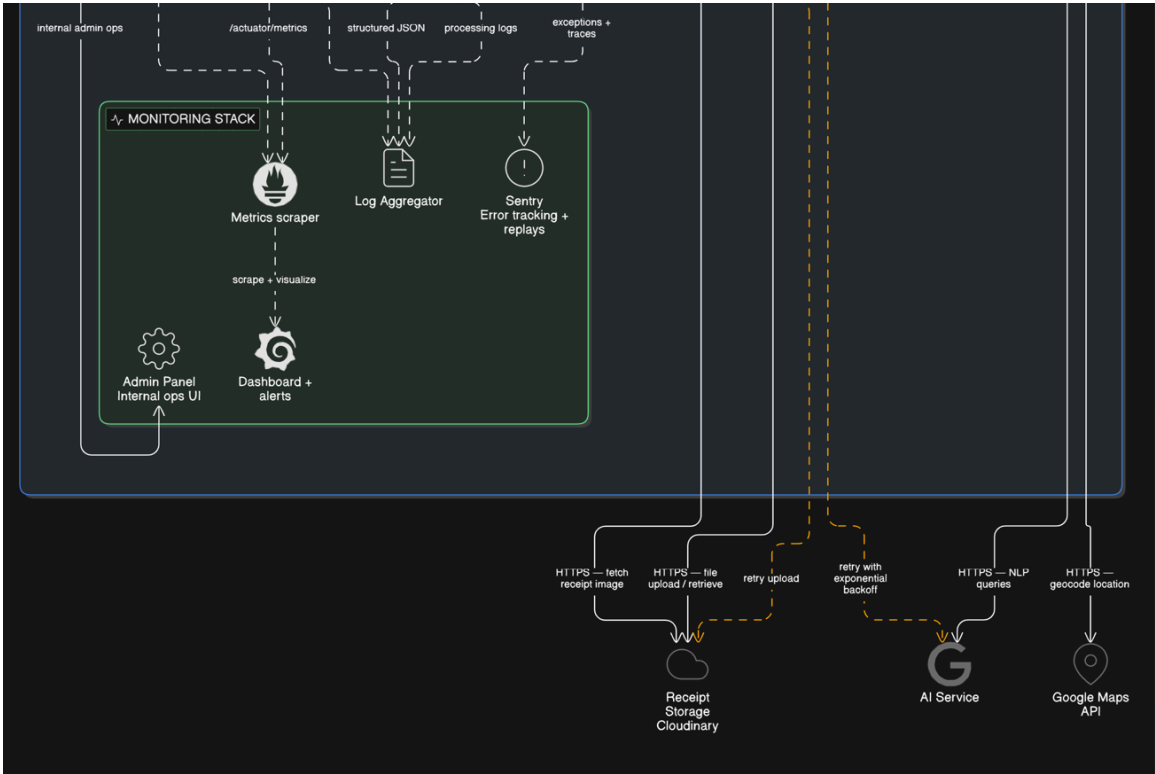
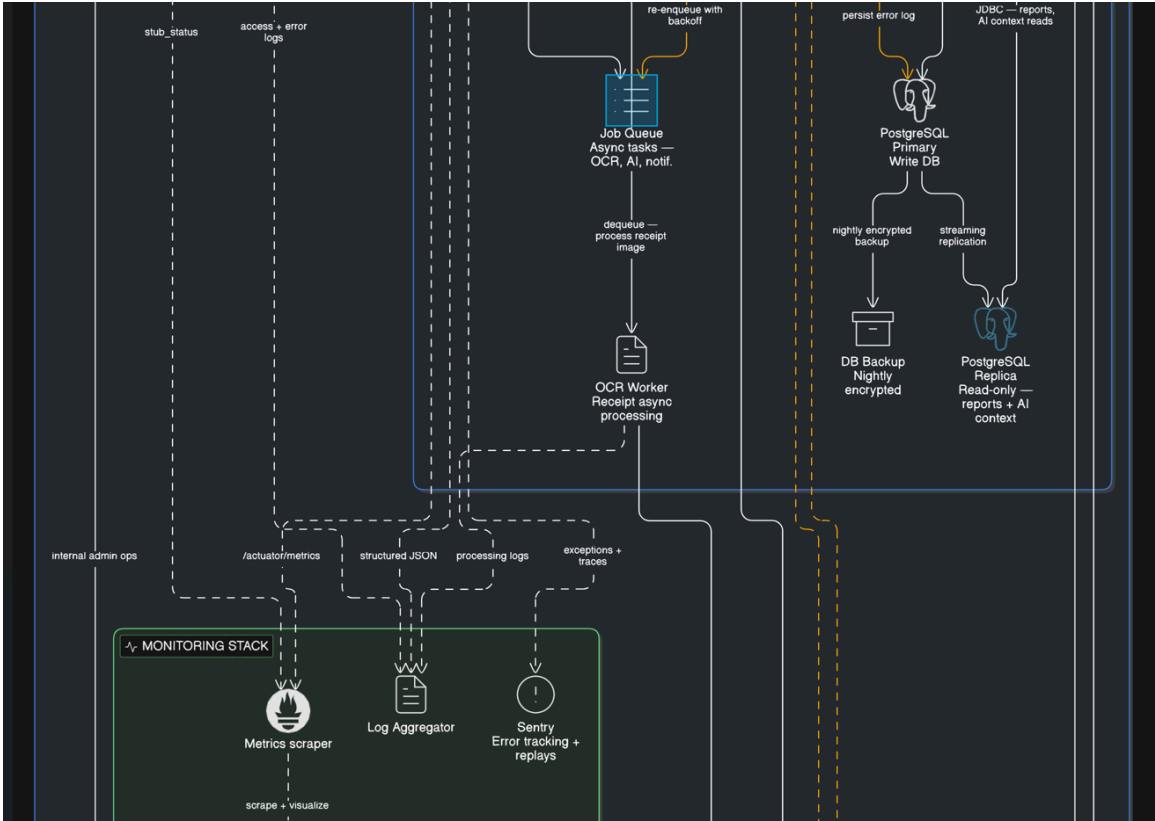
Container3: PostgreSQL(DB) - storage date

Sisteme externe: Gemini API, Cloudinary(stocare pdf,etc.), Google Maps API + Vercel sau ce vor folosi cei de la Frontend, momentan pune Frontend API si atat.

Flux: Frontend->Nginx->Spring Boot->PostgreSQL / API-uri
externe.







- Diagrame UML

A.Diagrama Use-Case - Sescu Tiberiu

Actori:

Utilizator

M4 Member

Developer

GitHub Actions

Certbot

Cloudinary

Description:

Utilizator:

-Utilizatorul aplicatiei

M4 Member:

-Membru al echipei care se ocupa de Hosting

Developer:

-Membru al unei echipe care se ocupa de celalte componente ale aplicatiei

GitHub Actions:

-Sistemul automatizat care execută pipeline-urile de testare (CI) și deployment (CD) la fiecare actualizare a codului.

Cloudinary:

-Serviciul extern de stocare în cloud utilizat pentru salvarea și gestionarea imaginilor (bonuri, extrase de cont) procesate de aplicație.

Certbot:

-Sistemul/autoritatea externă care emite și validează certificatele SSL necesare pentru securizarea traficului web (HTTPS).

Use cases:

Utilizator:

-Accesarea platformei de management al cheltuielilor=Utilizatorul introduce URL-ul aplicației în browser. Serverul (prin Nginx) îi returnează interfața grafică securizată, permițându-i să interacționeze cu sistemul pentru a-și vedea și adăuga cheltuielile zilnice.

M4 Member:

-Configurare VPS & Securitate=Crearea mașinii virtuale și setarea regulilor de bază de firewall.

-Gestionare Acces SSH=Securizarea accesului remote la server.

-Configurare Docker & Volume=Pregătirea mediului de containerizare și asigurarea persistenței datelor.

-Configurare Reverse Proxy (Nginx)=Instalarea și configurarea Nginx pentru a prelua traficul web și a-l direcționa către containerele interne.

-Securizare HTTPS= Criptarea traficului dintre utilizator și server pentru protejarea datelor financiare.

-Configurare Mediu de Stocare Extern (Cloudinary)=Pregătirea serviciului extern unde modulul OCR (M2) va salva pozele cu extrase de cont și bonuri.

Developer:

-Push cod pe GitHub=Dezvoltatorul trimite modificările de cod din mediul local în repository-ul central.

GitHub Actions:

-Verificarea testelor (CI)= Rularea automată a testelor la fiecare Push sau Pull Request pentru validarea codului.

-Build & Deploy Docker (CD)= Automatizarea procesului de construire a imaginii Docker și actualizarea acesteia pe VPS.

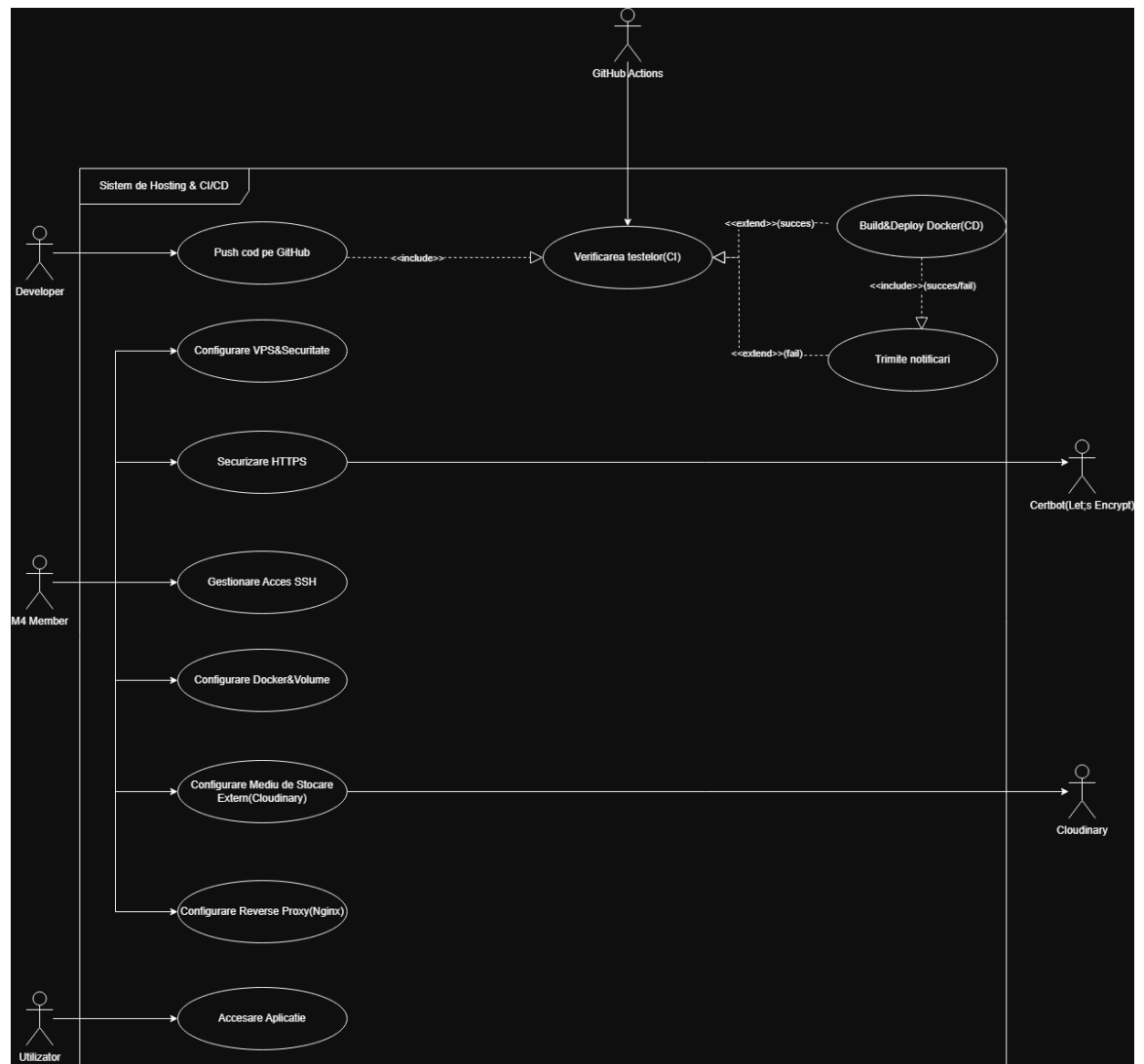
-Trimite notificări=Informarea echipei pe Discord/GitHub despre starea proceselor automate.

Certbot

-Emitere și Validare Certificat=Furnizează certificatele SSL necesare pentru conexiunea sigură.

Cloudinary

-Stocare Fișiere Media=Platforma externă care găzduiește efectiv imaginile procesate de aplicație



B.Diagrama de deployment(Specifica Devops) - Tarca Catalin

Deci se vor folosi cuburi pentru reprezentare.

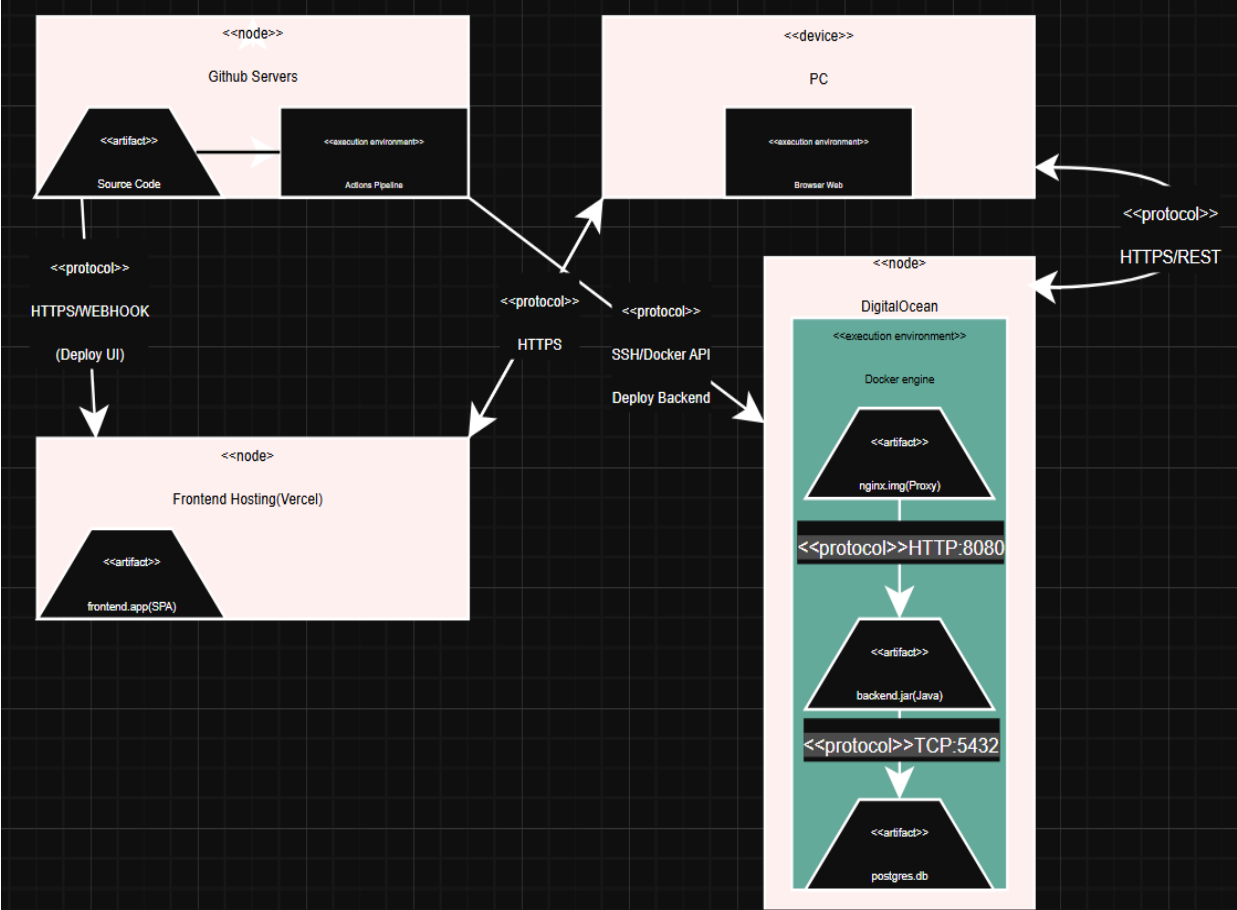
Avem cubul central DigitalOcean VPS, in interior se afla Docker Engine.

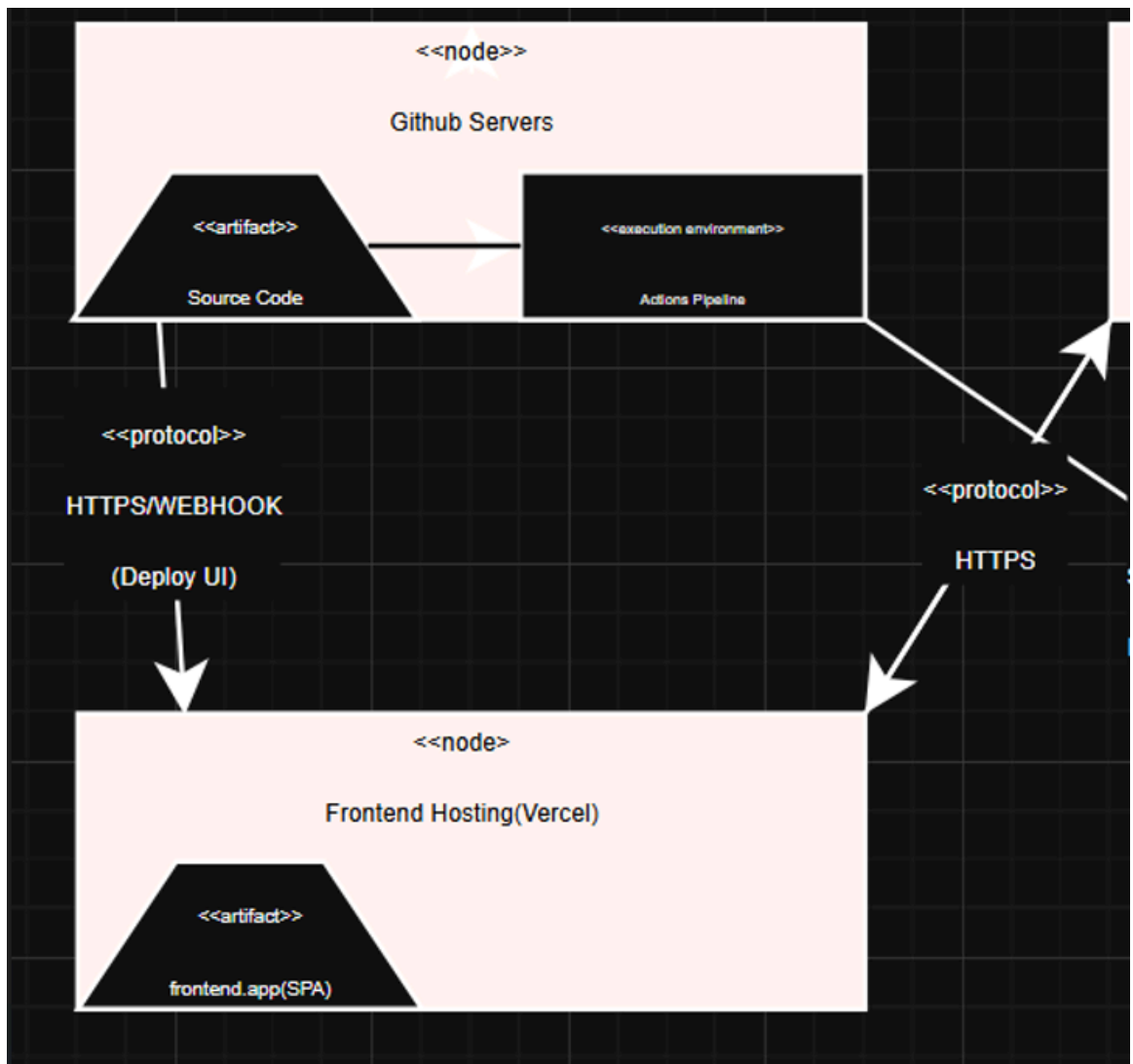
In Docker Engine vom avea 3 artefacte:nginx.img,backend.jar,[postgres.db](https://www.postgresql.org/)

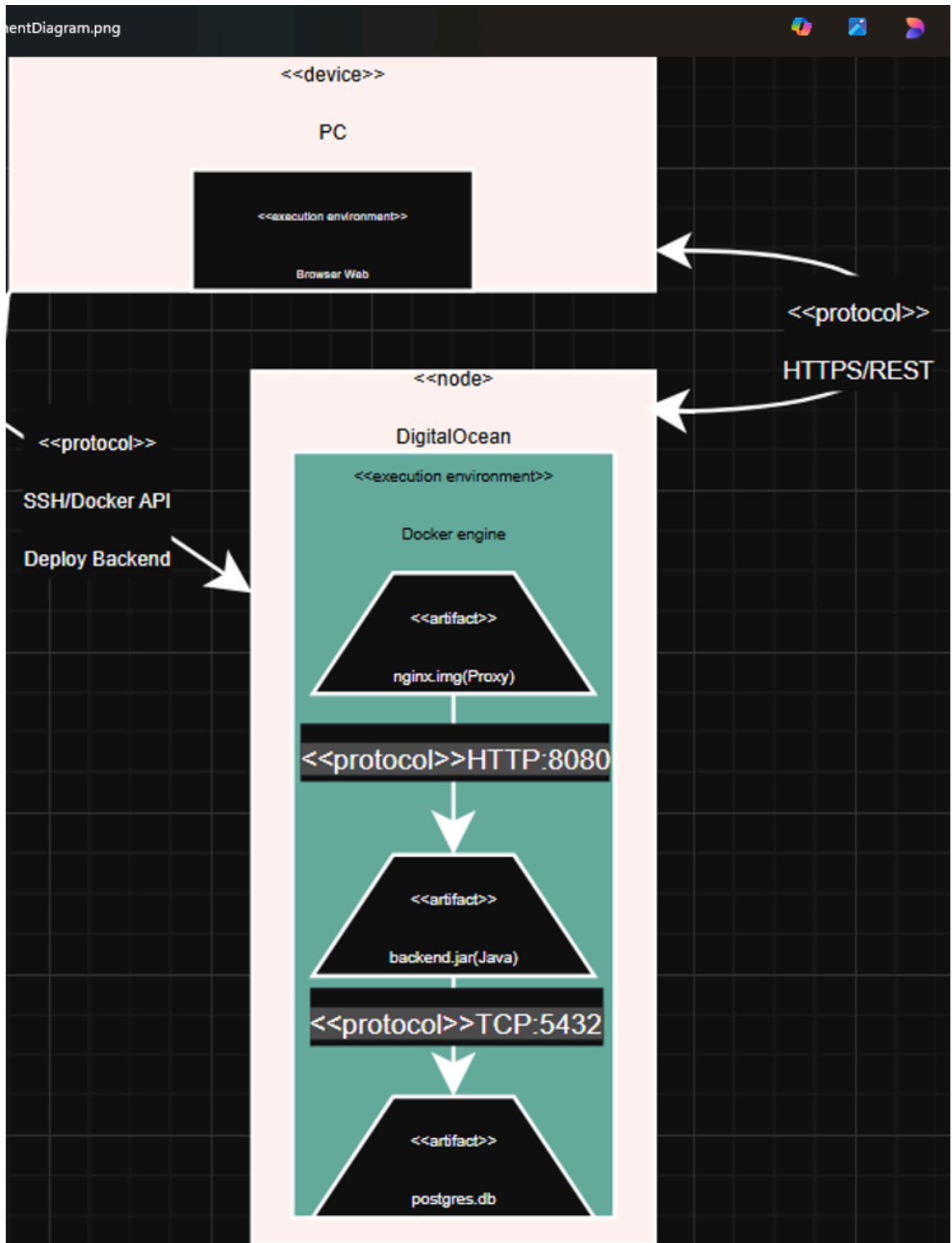
Nodul pentru Frontend unde va sta frontend.app

Nodul Github unde va sta source code si Actions Pipeline.

Linii intre noduri marcate cu protocolul folosit.(https/ssh,etc.)







C. Diagrama de flowline - Stupu Eduard

- reprezinti fluxul CI/CD(automatizarea)

Flux: Start->Push pe GitHub->Trigger Actiune->Rulare teste

Maven->Decizie, si aici ai 2 cazuri, DA: Build docker & deploy / NU: notificare Eroare -> END;

